UNIDAD 3 PARTE I EL MODELO RELACIONAL

1 INTRODUCCIÓN

1.1 Origen del modelo realcional

1.2 Objetivos de Codd

1.3 Las 12 reglas de Codd

2 ESTRUCTURA DEL MODELO RELACIONAL

2.1 Partes del modelo

2.2 Relación: esquema y extensión

2.3 Propiedades de una relación

2.4 Tipos de relaciones en un SGBD

2.5 Dominio

2.6 El valor NULL y la lógica trivalente

2.7 Operadores de tupla

2.8 Operadores sobre relaciones y su equivalente SQL

3 RESTRICCIONES DE INTEGRIDAD

3.1 Inherentes y semánticas

3.2 Claves candidatas, primarias y alternativas

3.3 Claves naturales y claves subrogadas

3.4 Integridad de entidad

3.5 Dominio y valor no nulo

3.6 Unicidad

3.7 Validación (CHECK)

3.8 Disparadores (triggers)

3.9 Integridad referencial: clave ajena

3.10 Mantenimiento de la integridad

4 EL MODELO RELACIONAL HOY

1. INTRODUCCIÓN

1.1. Origen del modelo relacional

El modelo entidad-relación (E/R) es un modelo **conceptual** que sirve para cualquier tipo de SGBD, en cambio, el modelo relacional es un modelo l**ógico** que sólo sirve para SGBD relacionales.

En la práctica. todos los diseñadores y administradores de bases de datos relacionales usan esquemas conceptuales entidad-relación porque se adaptan muy bien a este modelo y traducen las tablas de forma muy natural.

Hay que tener en cuenta la diferencia de la palabra **relación** en ambos modelos. En el modelo relacional una relación es una tabla mientras que en el entidad/relación es la asociación que se produce entre dos entidades (un alumno *se matrícula* en un módulo)

El modelo relacional fue propuesto en 1970, como alternativa a los modelos existentes, por el investigador **Edgar Codd,** cuando trabajaba en IBM**.** El modelo relacional es un modelo de datos basado en dos disciplinas matemáticas: **la lógica de predicados** y la **teoría de conjuntos**. Este modelo perseguía la sencillez de comprensión y de manipulación de la base de datos por parte del usuario final. Hasta entonces dominaban los modelos jerárquicos y en red (CODASYL), en los que el programador tenía que conocer los caminos de acceso a los datos.

Durante los años 70 fue un modelo teórico. En 1979 apareció Oracle, el primer SGBD relacional comercial con SQL, y en los 80 se impuso definitivamente. Hoy los SGBD relacionales más usados son:

SGBD	Licencia	Dónde se utiliza
Oracle Database	Comercial	Grandes empresas, banca, administración
MySQL	Libre (GPL) / comercial	Aplicaciones web, el que usaremos en clase
Microsoft SQL Server	Comercial	Empresas con tecnología Microsoft
PostgreSQL	Libre	Proyectos que necesitan funciones avanzadas, Intentaremos verlo este curso
SQLite	Dominio público	Móviles, navegadores, aplicaciones de escritorio
MariaDB	Libre	Bifurcación de MySQL, muy usada en Linux

Además, todos los proveedores de nube ofrecen estos SGBD como servicio gestionado: Amazon RDS y Aurora, Azure SQL Database o Google Cloud SQL. La empresa no instala ni mantiene el servidor, solo usa la base de datos.

1.2. Objetivos de Codd

- **Independencia física**. La forma de almacenar los datos, no debe influir en su manipulación lógica. Si el almacenamiento físico cambia (otro disco, un índice nuevo, otra organización de ficheros), los usuarios no tienen ni siquiera porque enterarse, seguirán funcionando sus aplicaciones.
- **Independencia lógica**. Se puede modificar el **esquema** de la base de datos (añadir una columna, añadir una tabla, dividir una tabla en dos) sin tener que modificar las aplicaciones ni las vistas que no usan esos cambios

- **Flexibilidad**. La base de datos ofrece fácilmente distintas vistas en función de los usuarios y aplicaciones.
- **Uniformidad**. Las estructuras lógicas siempre tienen una única forma conceptual (las tablas)
- **Sencillez**. El usuario no necesita saber programar caminos de acceso; le basta con decir *qué* datos quiere, no *cómo* obtenerlos.

Es notoria la importancia que Codd concede al tema de la **independencia de la representación lógica de los datos respecto a su almacenamiento interno** (independencia de **ordenación**, independencia de **indexación** e independencia de la **forma de acceso**)

El modelo relacional representa la segunda generación de los SGBD. En él, todos los datos están estructurados a nivel lógico como tablas formadas por filas y columnas, aunque a nivel físico puede tener una estructura completamente distinta. Un punto importante del modelo relacional es la sencillez de su estructura lógica. Pero, detrás de esa simple estructura hay un fundamento teórico matemático importante que hace que el modelo sea seguro y robusto.

1.3. Las 12 reglas de Codd (resumen)

En 1985 Codd publicó un conjunto de reglas (numeradas de 0 a 12) para decidir si un SGBD es realmente relacional. Ningún SGBD comercial las cumple todas, pero siguen siendo la referencia. Las más importantes para este módulo son:

- **Regla de la información:** todos los datos se representan como valores en tablas.
- **Acceso garantizado:** cualquier dato se alcanza con el nombre de la tabla, el valor de la clave primaria y el nombre de la columna.
- **Tratamiento sistemático de nulos:** existe un valor NULL, distinto de cero y de la cadena vacía, para la información desconocida o inaplicable.
- **Catálogo en línea relacional:** la descripción de la base de datos (metadatos) se guarda también en tablas. En MySQL es la base de datos `information_schema`.
- **Lenguaje de datos completo:** un lenguaje permite definir, manipular y controlar los datos. Hoy ese lenguaje es SQL.
- **Independencia física, lógica y de integridad:** las restricciones de integridad se definen en la base de datos, no en los programas.

2. ESTRUCTURA DEL MODELO RELACIONAL

2.1. Partes del modelo

El **modelo relacional** permite construir estructuras de datos para representar las diferentes informaciones del mundo real que tengan algún interés. Las estructuras de datos construidas siguiendo el modelo relacional están formadas por conjuntos de relaciones.

Las **relaciones** pueden ser concebidas como representaciones tabulares de los datos. La estructura del modelo relacional está formada por tres partes claramente diferenciadas:

- **Parte estructural**: Utiliza una estructura de datos muy sencilla, la relación.
- **Parte manipulativa**: Compuesta por **un conjunto de operadores** que permiten manejar la estructura anterior, el **Álgebra Relacional**. SQL es el lenguaje práctico que la implementa.

- La **Teoría de las Dependencias Funcionales y Normalización**: Compuesta por un conjunto de definiciones que permiten estudiar las dependencias entre los atributos de las relaciones y proporciona métodos para un correcto diseño de las bases de datos relacionales. Se estudiará en la Unidad 4.

2.2. Relación : esquema y extensión

La relación es el elemento básico del modelo relacional y está compuesta por dos partes:

1. **Esquema (cabecera o intensión)** Es la parte fija. Contiene el nombre de la relación, el nombre de sus atributos y el dominio de cada uno:
2. **Extensión (cuerpo).** Es la parte que cambia con el tiempo, el conjunto de tuplas (filas) que contiene la relación en un momento dado.

En el ámbito de las BD, podemos definir **tupla** como una secuencia finita de objetos que comprende las distintas asociaciones entre cada atributo de la relación y un valor concreto, admisible dentro del dominio respectivo.

Un **esquema de relación** es un conjunto de pares de la forma:

RELACIÓN $\{(A_1, D_1), (A_2, D_2), \ldots, (A_n, D_n)\}$

donde

- $\{A_1, A_2, \ldots, A_n\}$ es el conjunto de nombres de atributos (campos/columnas) del esquema, necesariamente distintos.
- $\{D_1, D_2, \ldots, D_n\}$ son los dominios asociados (tipos de datos) a dichos atributos, que no tienen que se necesariamente distintos.

Veamos un **ejemplo:**

Queremos guardar el catálogo de una tienda de videojuegos. De cada juego interesa el título, la plataforma, la clasificación PEGI y el precio

VIDEOJUEGO (título:dom_titulo, plataforma:dom_plataforma, pegi:dom_pegi, precio:dom_precio)

donde:

- {título, palataforma, pegi, precio} es el conjunto de nombres de atributos del esquema.
- Los dominios son:
  - dom_titulo: cadena de hasta 60 caracteres
  - dom_plataforma: {PC, PlayStaion, Xbox, Switch, Móvil}
  - dom_pegi: {3, 7, 12, 16, 18}
  - dom_precio: decimal con dos decimales, mayor o igual que 0

La extensión de una relación consiste en los valores de los datos almacenados en las tuplas que ésta contiene. Una tupla de esta relación es un conjunto de pares atributo–valor, donde cada valor pertenece a su dominio:

```
t1 = {(titulo, Galaxia Rota), (plataforma, PC), (pegi, 16), (precio, 29.99)}
```

La representación habitual es una tabla, con una fila por tupla y una columna por atributo.

titulo	plataforma	pegi	precio
Galaxia Rota	PC	16	29.99
Reino de Arena	Switch	7	49.99
Pixel Kart	Móvil	3	0.00

En MySQL, ese esquema se traduce así (las restricciones se explican en el apartado 3):

```
CREATE TABLE videojuego (
  titulo      VARCHAR(60)   NOT NULL,
  plataforma  ENUM('PC','PlayStation','Xbox','Switch','Móvil') NOT NULL,
  pegi        TINYINT       NOT NULL CHECK (pegi IN (3, 7, 12, 16, 18)),
  precio      DECIMAL(6,2)  NOT NULL CHECK (precio >= 0),
  PRIMARY KEY (titulo, plataforma)
);
```

2.3. Propiedades de una relación

- El **grado** es el número de atributos (columnas) que posee la relación. En el ejemplo del VIDEOJUEGO el grado es 4
- La **cardinalidad** se refiere al número de tuplas (filas) de la relación. En el ejemplo del VIDEOJUEGO la cardinalidad es 3. La cardinalidad cambia cada vez que insertamos o borramos.
- Además decimos que dos relaciones son compatibles, si sus esquemas son idénticos (mismo número de atributos con los mismos dominios). Es requisito indispensable para unirlas o restarlas.

Como una relación es un conjunto, cumple estas propiedades:

- Toda relación debe tener un nombre que la identifique unívocamente dentro de la BD.
- Cada fila está constituida por una tupla de datos relacionados entre ellos, que también podemos llamar *registro*, esta tupla guarda los datos que nos interesa reflejar de un objeto concreto del mundo real.
- No existen tuplas repetidas.
- El orden de las tuplas no importa.
- El orden de los atributos no importa.
- Cada atributo solo puede tomar un valor en cada tupla. Los valores de los atributos son atómicos, es decir no hay atributos multivaluados.
- Cada columna contiene, en cada celda, datos de un mismo tipo, es decir pertenecen al mismo dominio, podemos llamarla *atributo* o *campo*.

Esta tabla muestra una comparación de la terminología de una relación, una tabla y un fichero:

RELACIÓN	**TABLA**	**FICHERO**
Tupla	Fila	Registro
Atributo	Columna	Campo
Grado	Nº de columnas	Nº de campos
Cardinalidad	Nº de filas	Nº de registros
Modelo Relacional (teoría)	**SGBD Relacionales (implementación)**	**Sistemas de Ficheros**

Relación no es exactamente lo mismo que tabla. Una relación es un concepto matemático; una tabla de SQL es su implementación y se toma algunas licencias:

TABLA	RELACIÓN
Existe un orden entre sus filas, aunque no está garantizado sin `ORDER BY`	No existe un orden entre las tuplas, ya que una relación es un conjunto.
Existe un orden entre los elementos de de las filas (las columnas). Lo impone `CREATE TABLE`	Los pares de una tupla no están ordenados entre sí, ya que la tupla también es un conjunto.
Una tabla sin clave primaria ni `UNIQUE` admite filas duplicadas, y una consulta puede devolverlas si no se usa `DISTINCT`	No existen tuplas repetidas en una relación.
MySQL permite columnas `JSON` que guardan estructuras completas (ver apartado 4)	Es plana, es decir, que en el cruce de una fila y de una columna sólo puede haber un valor (no se admiten atributos multivaluados)

2.4. Tipos de relaciones en un SGBD

En un SGBD relacional pueden existir varios tipos de relaciones, aunque no todos manejan todos los tipos:

- **Tablas base:** tienen nombre y sus datos están almacenados. Se crean con `CREATE TABLE`.
- **Vistas:** relaciones virtuales. Tienen nombre, pero no datos propios: se definen con una consulta sobre otras tablas y se calculan cada vez que se usan (`CREATE VIEW`).
- **Vistas materializadas:** como las vistas, pero el resultado se almacena y se refresca periódicamente. Aceleran consultas pesadas. Las tienen Oracle y PostgreSQL; MySQL no las implementa de forma nativa (se simulan con una tabla y un evento programado).
- **Resultados de consultas:** la relación que devuelve un `SELECT`. No persiste en la base de datos.
- **Resultados intermedios:** los de las subconsultas y las CTE (`WITH`). Tampoco persisten.
- **Tablas temporales:** tienen nombre y datos, pero se destruyen solas al cerrar la sesión (`CREATE TEMPORARY TABLE`).

2.5. Dominio

Un dominio es el conjunto de valores **homogéneos** (todos del mismo tipo) y **atómicos** (indivisibles para el modelo) que puede tomar un atributo. Tiene un nombre, un tipo de datos y, opcionalmente, unidades y restricciones.

Un dominio es más que un tipo de datos. El tipo de `pegi` es un entero, pero su dominio es solo {3, 7, 12, 16, 18}. Un teléfono puede guardarse como `CHAR(9)`, pero su dominio son las cadenas de 9 dígitos, no cualquier texto de 9 caracteres.

El estándar SQL tiene `CREATE DOMAIN` y PostgreSQL lo implementa. **MySQL no lo tiene**, así que el dominio se construye combinando:

- El tipo de datos: `TINYINT`, `VARCHAR(60)`, `DECIMAL(6,2)`, `DATE`…

- `ENUM` para listas cerradas de valores.
- `CHECK` para el resto de condiciones: `CHECK (telefono REGEXP '^[0-9]{9}$')`.

2.6. El valor NULL y la lógica trivalente

Como se puede observar el dominio se asemeja a un tipo de datos, pero tiene muchas más connotaciones que este término, como que incluye el valor NULO.

El valor NULO en el Modelo Relacional es ausencia de información: el valor es desconocido o no se aplica. No es cero, ni la cadena vacía, ni un espacio.

Por eso **NULL no se puede comparar con =**. Cualquier comparación con NULL no da verdadero ni falso, sino **desconocido** (UNKNOWN). SQL trabaja con lógica de tres valores:

A	B	A AND B	A OR B	NOT A
TRUE	UNKNOWN	UNKNOWN	TRUE	FALSE
FALSE	UNKNOWN	FALSE	UNKNOWN	TRUE
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN

Un `WHERE` solo devuelve las filas en las que la condición es `TRUE`; las UNKNOWN se descartan. Consecuencias que conviene recordar:

- `WHERE precio = NULL` no devuelve nunca nada. Lo correcto es `WHERE precio IS NULL`.
- `WHERE pegi <> 18` no devuelve las filas con pegi NULL.
- `NOT IN (subconsulta)` devuelve cero filas si la subconsulta contiene algún NULL. Es una fuente frecuente de errores; `NOT EXISTS` no tiene este problema.

2.7. Operadores de tupla

Dada una tupla t, hay dos operaciones básicas:

Operación	Notación de punto	Equivalente SQL
Consultar el valor de un atributo	`t.precio`	`SELECT precio FROM videojuego WHERE …`
Asignar un valor a un atributo	`t.precio ← 24.99`	`UPDATE videojuego SET precio = 24.99 WHERE …`

Si `t3.precio` no tiene valor, decimos que *t3.precio es nulo* (`IS NULL`), nunca que *t3.precio = NULL*.

2.8. Operadores asociados a la estructura relación y su equivalente SQL

Una relación es un conjunto de tuplas, así que se manipula con operaciones de conjuntos. Estas operaciones forman el álgebra relacional, y cada una tiene su traducción en SQL:

Operador	Qué hace	SQL (MySQL 8 o superior)
Inserción	Añade una tupla	`INSERT`
Borrado	Elimina tuplas	`DELETE`
Selección (σ)	Se queda con las tuplas que cumplen una condición	`WHERE`
Proyección (π)	Se queda con algunos atributos	Lista de columnas del `SELECT`

Operador	Qué hace	SQL (MySQL 8 o superior)
Unión (∪)	Tuplas que están en una u otra relación	`UNION`
Diferencia (−)	Tuplas de la primera que no están en la segunda	`EXCEPT` (desde 8.0.31) o `NOT EXISTS`
Intersección (∩)	Tuplas que están en las dos	`INTERSECT` (desde 8.0.31) o `EXISTS`
Producto cartesiano (×)	Combina cada tupla de una con cada tupla de otra	`CROSS JOIN`
Combinación o join (⋈)	Producto cartesiano + selección por atributos comunes	`INNER JOIN … ON`

Unión, diferencia e intersección exigen relaciones **compatibles**. El join es el operador que más usarás: es la forma de recuperar datos que están repartidos en varias tablas relacionadas mediante claves ajenas.

3. RESTRICCIONES DE INTEGRIDAD

3.1. Inherentes y semánticas

Una restricción de integridad es una condición que los datos deben de cumplir siempre. Hay dos tipos:

- **Inherentes:** vienen impuestas por el propio modelo relacional y no hace falta declararlas. Las más importantes son:
  - No puede haber dos tuplas iguales
  - El orden de las tuplas no es significativo
  - El orden de los atributos no es significativo
  - Cada atributo sólo puede tomar un valor en el dominio en el que está inscrito, es decir, los atributos son atómicos y homogéneos.
  - Toda relación tiene clave primaria, y ningún atributo que forma parte de la clave primaria puede ser nulo (**regla de integridad de entidad**)
  - Una clave ajena sólo puede contener valores que existen en la clave referenciada, o NULL (**regla de integridad referencial**)
- **Semánticas:** definidas por el usuario. Dependen del problema que modelamos y las declara el diseñador. Son las que reflejan las reglas del mundo real: qué atributo es la clave, qué campos son obligatorios, qué valores son válidos.

La frontera entre ambas se entiende así: que toda tabla tenga clave primaria lo exige el modelo; cuál es esa clave lo decide el diseñador según el problema.

Resumen de restricciones semánticas y su sintaxis en MySQL:

Restricción	Qué garantiza	MySQL
Clave primaria	Identifica cada fila; única y no nula	`PRIMARY KEY`
Valor no nulo	El atributo es obligatorio	`NOT NULL`
Unicidad	No se repiten valores	`UNIQUE`
Validación	El valor cumple una condición	`CHECK`
Valor por defecto	Valor si no se indica otro	`DEFAULT`
Clave ajena	El valor existe en otra tabla	`FOREIGN KEY … REFERENCES`
Disparador	Regla compleja programada	`CREATE TRIGGER`

3.2. Claves candidatas, primaria y alternativas

Una **clave candidata** es un conjunto de atributos que identifican unívoca y mínimamente cada tupla de la relación. De la propia definición de relación se deriva que siempre existe, al menos, una clave candidata (al ser una relación un conjunto y no existir dos tuplas iguales, el conjunto de todos los atributos siempre tiene que identificar unívocamente a cada tupla). La propiedad de minimalidad implica que no se incluye ningún atributo innecesario .

Si una relación tiene varias claves candidatas, entonces se debe distinguir entre:

- **Clave Primaria** (Primary Key):
  - Es la clave candidata que el usuario escoge para identificar las tuplas de la relación.
  - Cuando sólo existe una clave candidata, ésta es la clave primaria (siempre existe clave primaria).
- **Claves Alternativas** (Alternative Key): Las claves candidatas que no han sido escogidas como clave primaria.

Ejemplo. En una tabla ALUMNO con nia (número de identificación del alumno), dni y email corporativo, las tres son claves candidatas. Elegimos nia como primaria; dni y email son alternativas.

La restricción de clave primaria implica dos cosas:

- **Unicidad:** sus valores no se repiten.
- **Valor no nulo:** ningún atributo de la clave puede ser NULL, tampoco en una clave compuesta.

Una clave alternativa debe declararse como `UNIQUE` y `NOT NULL`. Solo con `UNIQUE` no basta.

Ejemplo:

```
CREATE TABLE alumno (
       nia            INT UNSIGNED   NOT NULL,
       dni            CHAR(9)        NOT NULL,
       email          VARCHAR(100)   NOT NULL,
       nombre         VARCHAR(100)   NOT NULL,
       apellidos      VARCHAR(100)   NOT NULL,
       PRIMARY KEY (nia),
       CONSTRAINT uq_alumno_dni UNIQUE (dni),
       CONSTRAINT uq_alumno_email UNIQUE (email)
```

```
);
```

Dar nombre a las restricciones con CONSTRAINT es una buena práctica: los mensajes de error lo indican, y luego se pueden eliminar por su nombre.

3.3. Claves naturales y claves subrogadas

- **Clave natural:** un atributo con significado en el mundo real (DNI, ISBN, matrícula).
- **Clave subrogada:** un identificador artificial sin significado, generado por el sistema. En MySQL, normalmente `INT AUTO_INCREMENT`; en sistemas distribuidos, un UUID.

Hoy es habitual usar claves subrogadas como primaria y declarar las naturales como alternativas (UNIQUE NOT NULL). Motivos:

- Las claves naturales a veces **cambian** o **no existen** para todos (no todo el mundo tiene DNI español).
- Son **más largas**, y la clave primaria se copia en cada clave ajena que la referencia.
- Usar datos personales como el DNI de clave primaria los **repite por toda la base de datos**. Eso choca con el principio de minimización de datos del RGPD.

3.4. Integridad de entidad

La regla de integridad de entidad es el mecanismo que garantiza la identificación y unicidad de las tuplas en una relación.

¿Qué dice esta regla? Ningún atributo que forme parte de la clave primaria de una relación puede tomar un valor nulo.

De esta manera nos aseguramos que no se pueden repetir dos tuplas en una relación. Veamos que esto es así tanto si la clave primaria es un único atributo, o si está compuesta por más de un atributo.

Clave primaria simple. En esta tabla los nombres se guardan como inicial y apellidos. Si se permitiera dni nulo, las dos últimas filas serían indistinguibles, aunque correspondan a dos hermanos:

dni	**nombre**	**ciclo**
23456987K	S. López Ojeda	DAM1
34562358B	D. Rodríguez Gracia	DAM1
NULL	A. Bonillo Sierra	SMR1
NULL	A. Bonillo Sierra	SMR1

¿Cuál de las dos filas es Alfonso y cuál es Andrés? Sin un valor de clave no hay forma de saberlo, ni de modificar o borrar solo una de ellas.

Clave primaria compuesta. En una tabla PEDIDO con dni_cliente, cod_tienda, fecha_pedido e importe, podría pensarse que la clave es (dni_cliente, cod_tienda, fecha_pedido). Pero un cliente puede hacer dos pedidos el mismo día en la misma tienda, y esa clave no podría registrarlos. La solución habitual es una clave subrogada:

```
CREATE TABLE pedido (
  id_pedido     INT UNSIGNED  NOT NULL AUTO_INCREMENT,
  dni_cliente   CHAR(9)       NOT NULL,
```

```
  cod_tienda    CHAR(3)       NOT NULL,
  fecha_pedido  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  importe       DECIMAL(8,2)  NOT NULL,
  PRIMARY KEY (id_pedido)
);
```

Las claves compuestas siguen siendo correctas y necesarias en otros casos, por ejemplo en las tablas que resuelven relaciones N:N: en MATRICULA (nia, cod_modulo, curso_academico) esa combinación sí es única por naturaleza.

3.5. Dominio y valor no nulo

- **Restricción sobre dominio:** El asociar un dominio a cada atributo restringe el conjunto de valores que puede tomar ese atributo. En MySQL se aplica con el tipo de datos, `ENUM` y `CHECK`
- **Restricción de valor no nulo u obligatoriedad:** La definición de una restricción de valor no nulo sobre un conjunto de atributos K de la relación R expresa la siguiente propiedad: "no debe haber en R una tupla que tenga el valor nulo en algún atributo de K". Es decir, no se admiten valores nulos.

3.6. Unicidad

Impide que los valores de los atributos marcados de esa forma, puedan repetirse. Esta restricción debe indicarse en todas las claves alternativas o candidatas. En MySQL usamos `UNIQUE`.

Matiz importante en MySQL: una columna `UNIQUE` **admite varios NULL**, porque NULL no es igual a NULL. Si dos alumnos tienen email NULL, MySQL no lo considera repetido. Por eso una clave alternativa necesita `UNIQUE` y `NOT NULL` a la vez.

3.7. Validación (CHECK)

`CHECK` comprueba, en cada inserción o modificación, que el valor cumple una condición. Si no la cumple, rechaza la operación.

```
CREATE TABLE contrato (
  id_contrato   INT UNSIGNED  NOT NULL AUTO_INCREMENT PRIMARY KEY,
  sueldo        DECIMAL(8,2)  NOT NULL,
  fecha_inicio  DATE          NOT NULL,
  fecha_fin     DATE,
  CONSTRAINT ck_sueldo CHECK (sueldo > 1000),
  CONSTRAINT ck_fechas CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio)
);
```

Cosas a tener en cuenta en MySQL:

- MySQL ignoraba los `CHECK` hasta la versión 8.0.16: los aceptaba pero no los comprobaba. En versiones antiguas (o en scripts antiguos) no te fíes de ellos.
- Un `CHECK` no puede usar funciones no deterministas como `CURDATE()` o `NOW()` (error 3814). Para comprobar algo como la fecha de nacimiento no puede ser futura hay que usar un disparador.
- Una condición `CHECK` que da `UNKNOWN` (porque algún valor es `NULL`) se considera cumplida. Por eso en el ejemplo se contempla explícitamente `fecha_fin IS NULL`.

3.8. Disparadores (Triggers)

Un disparador es un pequeño programa almacenado en la base de datos que se ejecuta automáticamente cuando ocurre un evento sobre una tabla (`INSERT`, `UPDATE` o `DELETE`), antes (`BEFORE`)

o después (AFTER) de la operación. Permiten expresar reglas que no caben en las demás restricciones.

```
DELIMITER //
CREATE TRIGGER tr_alumno_fecha_nac
BEFORE INSERT ON alumno_ficha
FOR EACH ROW
BEGIN
  IF NEW.fecha_nacimiento > CURDATE() THEN
    SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'La fecha de nacimiento no puede ser futura';
  END IF;
END//
DELIMITER ;
```

Son muy potentes, pero también los más difíciles de mantener: la lógica queda "escondida" en la base de datos. La recomendación actual es usar primero las restricciones declarativas (`NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY`) y reservar los disparadores para lo que estas no pueden expresar. Se estudian en detalle en la unidad de programación de bases de datos.

3.9. Integridad referencial: clave ajena

Se denomina **clave ajena** de una relación R2 a un conjunto no vacío de atributos cuyos valores han de coincidir con los valores de la clave candidata de una relación R1, normalmente la primaria. Una clave ajena está constituida por un atributo, o por un conjunto de atributos, del esquema de una relación, que sirve para relacionar sus tuplas con los tuplas de otra relación de la base de datos (o con las tuplas de ella misma, en algunos casos)

- R1 y R2 pueden ser la misma relación (relación reflexiva: un empleado y su jefe)
- La clave ajena y la correspondiente clave candidata referenciada han de estar definidas sobre el mismo dominio. En MySQL, mismo tipo de datos (y mismo signo en los enteros).
- La clave ajena puede ser NULL, salvo que se declare `NOT NULL`. Un NULL significa "sin asociación".

El uso de claves ajenas es el mecanismo que proporciona el modelo relacional para expresar asociaciones entre los objetos representados en la base de datos.

Regla de integridad referencial: una clave ajena solo puede contener valores que existan en la clave referenciada, o NULL.

Ejemplo. En una plataforma de alquiler de patinetes, cada fila de ALQUILER tiene un id_cliente que referencia a CLIENTE. Preguntas para pensar:

- ¿Qué pasaría si se insertara un alquiler con un id_cliente que no existe?
- ¿Qué hacemos con sus alquileres si un cliente da de baja su cuenta?
- ¿Los borramos todos? ¿Los conservamos sin cliente? ¿Impedimos la baja mientras tenga alquileres?

Lo primero lo impide directamente la integridad referencial. Para lo segundo, al definir la clave ajena se elige una **política** para cuando se borra (ON DELETE) o se modifica (ON UPDATE) la fila referenciada:

Política	Qué hace	En MySQL (InnoDB)
`RESTRICT`	Prohíbe borrar o modificar la fila si tiene	Soportada

	filas que la referencian	
`NO ACTION`	En el estándar, igual que RESTRICT pero comprobando al final de la sentencia	Soportada; en MySQL se comporta exactamente igual que `RESTRICT`. Es la política por defecto
`CASCADE`	Propaga la operación: borra o modifica también las filas que la referencian	Soportada
`SET NULL`	Pone a NULL la clave ajena de las filas que la referencian	Soportada; la columna no puede ser `NOT NULL`
`SET DEFAULT`	Pone el valor por defecto en la clave ajena	MySQL la reconoce, pero **InnoDB la rechaza**.

```
CREATE TABLE alquiler (
  id_alquiler  INT UNSIGNED  NOT NULL AUTO_INCREMENT PRIMARY KEY,
  id_cliente   INT UNSIGNED,
  id_patinete  INT UNSIGNED  NOT NULL,
  inicio       DATETIME      NOT NULL,
  CONSTRAINT fk_alquiler_cliente FOREIGN KEY (id_cliente)
    REFERENCES cliente (id_cliente)
    ON DELETE SET NULL
    ON UPDATE CASCADE,
  CONSTRAINT fk_alquiler_patinete FOREIGN KEY (id_patinete)
    REFERENCES patinete (id_patinete)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);
```

Con esta definición, si un cliente se da de baja sus alquileres se conservan (para la contabilidad) sin cliente asociado. En cambio, no se puede borrar un patinete que tenga alquileres registrados.

3.10. Mantenimiento de la integridad

Hemos visto la importancia de la integridad en el Modelo Relacional, y es por tanto necesario mantenerla en todo momento. ¿Cómo mantenemos dicha integridad? Gracias a un principio básico, la integridad ha de ser mantenida en todo momento por el sistema.

Veamos cómo se logra tal objetivo considerando las dos reglas de integridad más importantes:

1. Integridad de entidad: Se debe comprobar que los atributos que forman parte de una clave primaria son no nulos y que el valor de dichos atributos no se repite en los procesos de inserción y actualización.
2. Integridad referencial:

   1. En inserción, comprobar que el valor de la clave ajena es nula o coincide con un valor existente de la clave primaria de la tabla que referencia.

   2. En actualización, si se actualiza la clave ajena, comprobar las condiciones que definen la clave ajena (ver si el nuevo valor existe como clave primaria en la relación referenciada), y si se actualiza la clave primaria, actualizar en cadena la clave ajena.

   3. En borrado, si se borra la clave primaria, borrar en cascada o poner a null la clave ajena que

hace referencia a dicha clave primaria, es decir, si eliminamos un registro de la tabla R1 referenciada, se eliminan en cascada todos los registros de la tabla R2 que hacen referencia al registro de R1, o se ponen a null las claves ajenas de R2 que referencia a dicha clave primaria.

Decimos que se produce un borrado en cascada porque al borrar los registros de la tabla R2 puede hacerse posible borrar más registros de una tabla R3 que referenciaran a esos registros de R2, y así sucesivamente a lo largo de toda la cadena de referencias que existieran.

Cuando la política es `CASCADE`, el efecto puede encadenarse: al borrar filas de R2 se pueden borrar a su vez filas de R3 que las referencian, y así sucesivamente. Por eso `ON DELETE CASCADE` debe usarse con cuidado: un solo `DELETE` puede eliminar datos de muchas tablas.

4. EL MODELO RELACIONAL EN LA ACTUALIDAD

Más de cincuenta años después, el modelo relacional sigue siendo el más utilizado. La mayoría de los SGBD más populares del mundo son relacionales. Pero ya no es la única opción, y conviene saber cuándo se elige otra.

Bases de datos NoSQL. A partir de 2009 se popularizaron SGBD que renuncian a parte del modelo relacional (esquema fijo, joins, a veces transacciones completas) a cambio de escalar con facilidad en muchos servidores o de guardar datos poco estructurados:

Tipo	Cómo guarda los datos	Ejemplo	Uso típico
Documental	Documentos JSON con estructura flexible	MongoDB	Catálogos de productos, perfiles de usuario
Clave-valor	Pares clave → valor, muy rápidos	Redis	Cachés, sesiones web
Columnar	Familias de columnas distribuidas	Cassandra	Grandes volúmenes de escrituras (sensores, registros)
Grafos	Nodos y aristas	Neo4j	Redes sociales, recomendaciones

Los relacionales también han evolucionado. Los SGBD relacionales actuales han incorporado ideas de NoSQL:

- **Columnas JSON.** MySQL tiene el tipo JSON desde la versión 5.7. Permite guardar en una columna un documento completo, por ejemplo las características variables de un producto.
- **Columnas vectoriales.** MySQL 9 incorpora el tipo VECTOR, pensado para aplicaciones de inteligencia artificial (búsqueda por similitud).
- **NewSQL.** SGBD como CockroachDB o Google Spanner mantienen SQL y el modelo relacional, pero se distribuyen en muchos servidores como un NoSQL.

¿Una columna JSON rompe el modelo relacional? Una columna JSON contiene valores no atómicos, así que se salta una de las propiedades de la relación. A cambio, da flexibilidad cuando los atributos varían mucho de un elemento a otro. La recomendación práctica: los datos que se consultan, se filtran o se relacionan con otras tablas van en columnas normales; JSON solo para información secundaria y variable.

¿Cuándo se elige cada uno? Un modelo relacional encaja cuando los datos tienen estructura clara, hay muchas relaciones entre ellos y la integridad es crítica: matrículas, facturación, banca, reservas. Un NoSQL encaja cuando la estructura cambia mucho, el volumen es enorme o se prioriza la velocidad sobre la consistencia inmediata. En muchas empresas conviven los dos.